Análisis comparativo del rendimiento de los algoritmos Insertion Sort y QuickSort bajo diferentes patrones de entrada

Comparative Analysis of the Performance of Insertion Sort and QuickSort Algorithms Under Different Input Patterns

Matias Fabricio Copaja Segura

mcopajas@unjbg.edu.pe

Facultad de Ingeniería, Universidad Nacional Jorge Basadre Grohmann

Tacna, Tacna — Perú

Resumen: *El presente trabajo analiza comparativamente el rendimiento de los algoritmos de ordenamiento Insertion Sort y QuickSort mediante una evaluación experimental implementada en C++. Se utilizaron conjuntos de 1 000 y 10 000 elementos con cinco patrones de entrada: aleatorio, ascendente, descendente, casi ordenado y con valores duplicados. Para cada combinación de algoritmo, tamaño y patrón se realizaron 30 repeticiones, registrando el tiempo promedio de ejecución, las comparaciones, los intercambios, los movimientos y la profundidad máxima de recursión. Los resultados mostraron que QuickSort obtuvo menores tiempos promedio en ocho de las diez combinaciones evaluadas. En particular, para 10 000 elementos en orden descendente, fue aproximadamente 237 veces más rápido que Insertion Sort. Sin embargo, Insertion Sort presentó un mejor rendimiento con entradas ordenadas ascendentemente, donde fue aproximadamente 9,8 veces más rápido para el mismo tamaño. Se concluye que el rendimiento de ambos algoritmos depende del tamaño y del patrón de entrada, por lo que la selección del método de ordenamiento debe considerar las características de los datos y las condiciones de aplicación.*

Palabras clave: Algoritmos de ordenamiento, Insertion Sort, QuickSort, análisis de rendimiento, complejidad algorítmica.

Abstract: *This study experimentally compares the performance of the Insertion Sort and QuickSort sorting algorithms using C++. Input datasets containing 1,000 and 10,000 elements were evaluated under five patterns: random, ascending, descending, nearly sorted, and duplicate values. Thirty repetitions were performed for each combination of algorithm, input size, and pattern. The recorded metrics included average execution time, comparisons, swaps, movements, and maximum recursion depth. The results showed that QuickSort achieved lower average execution times in eight of the ten evaluated combinations. For 10,000 elements in descending order, QuickSort was approximately 237 times faster than Insertion Sort. However, Insertion Sort performed better on ascending inputs, being approximately 9.8 times faster for the same input size. The findings indicate that the performance of both algorithms depends on input size and data pattern; therefore, algorithm selection should consider the characteristics of the input and the requirements of the application.*

Keywords: Sorting algorithms, Insertion Sort, QuickSort, performance analysis, algorithmic complexity.

1 Introducción

El ordenamiento de datos es una operación fundamental en el desarrollo de software, debido a que permite organizar elementos según un criterio determinado y facilita el procesamiento posterior de la información. La eficiencia de esta operación depende, entre otros factores, de la cantidad de elementos que deben ordenarse, de su distribución inicial y del algoritmo empleado. Por ello, analizar el comportamiento de diferentes algoritmos de ordenamiento permite comprender sus ventajas y limitaciones en distintas condiciones de ejecución.

Entre los algoritmos de ordenamiento más conocidos se encuentran Insertion Sort y QuickSort. Insertion Sort construye progresivamente una secuencia ordenada mediante la inserción de cada elemento en su posición correspondiente. Su implementación es sencilla y

presenta un rendimiento favorable cuando los datos ya están ordenados o tienen pocas alteraciones. Sin embargo, su complejidad temporal en el caso promedio y en el peor caso es cuadrática, (O(n^2)). Por su parte, QuickSort utiliza una estrategia de partición y llamadas recursivas para ordenar los elementos. Su rendimiento temporal promedio es (O(nlogn)), aunque puede alcanzar (O(n^2)) en el peor caso, dependiendo del comportamiento de las particiones (Massachusetts Institute of Technology [MIT], s. f.; Cornell University, 2026).

La comparación entre ambos algoritmos resulta especialmente relevante porque sus diferencias no se limitan a la complejidad teórica. El orden inicial de los elementos puede influir significativamente en el número de comparaciones, movimientos e intercambios realizados durante la ejecución. Por esta razón, una evaluación experimental permite observar cómo se manifiestan dichas diferencias en condiciones concretas y complementar el análisis teórico con mediciones obtenidas mediante la ejecución de los programas.

En este contexto, el presente trabajo tiene como objetivo analizar comparativamente el rendimiento de Insertion Sort y QuickSort mediante su implementación en C++. Para ello, se consideran conjuntos de 1 000 y 10 000 elementos, organizados según cinco patrones de entrada: aleatorio, ascendente, descendente, casi ordenado y con valores duplicados. La evaluación contempla el tiempo de ejecución, las comparaciones, los intercambios, los movimientos y, en el caso de QuickSort, la profundidad máxima de recursión registrada por la implementación.

Finalmente, los resultados experimentales se analizan mediante medidas estadísticas y representaciones gráficas, con el propósito de identificar las condiciones en las que cada algoritmo presenta un mejor rendimiento y contrastar las observaciones con sus características teóricas.

2 Fundamentos teóricos

2.1 Algoritmos de ordenamiento

Los algoritmos de ordenamiento permiten organizar un conjunto de elementos según un criterio establecido, como el orden numérico ascendente. Su rendimiento puede evaluarse mediante el tiempo de ejecución, la cantidad de operaciones efectuadas y los recursos computacionales utilizados. Estas características permiten comparar distintas soluciones para un mismo problema y determinar en qué condiciones resulta conveniente cada una (Massachusetts Institute of Technology [MIT], s. f.).

En este estudio se analizan Insertion Sort y QuickSort, dos algoritmos que emplean estrategias diferentes para ordenar los elementos. El primero construye progresivamente una secuencia ordenada, mientras que el segundo divide el conjunto mediante un elemento pivote y ordena recursivamente las particiones obtenidas.

2.2 Insertion Sort

Insertion Sort ordena los elementos recorriendo el conjunto desde la segunda posición. En cada iteración, toma un elemento y lo inserta en la posición correspondiente dentro de la parte previamente ordenada, desplazando hacia la derecha los elementos que son mayores que él.

Una de sus principales ventajas es su sencillez y su capacidad para aprovechar entradas que ya se encuentran ordenadas o presentan pocas alteraciones. Cuando los elementos están en orden ascendente, el algoritmo realiza un trabajo mínimo en cada iteración. En cambio, si los datos están en orden descendente, debe desplazar numerosos elementos para insertar cada nueva clave.

Su complejidad temporal en el mejor caso es (O(n)), mientras que en los casos promedio y peor es (O(n^2)). Su espacio auxiliar es (O(1)), ya que la implementación utiliza una cantidad constante de variables adicionales (Cornell University, 2026).

2.3 QuickSort

QuickSort aplica la estrategia de divide y vencerás. Primero selecciona un elemento denominado pivote y divide el conjunto en dos regiones según la relación de sus elementos con dicho pivote. Posteriormente, aplica recursivamente el mismo procedimiento a las regiones obtenidas hasta completar el ordenamiento.

El rendimiento de QuickSort depende de la calidad de las particiones. Cuando estas dividen el conjunto de manera relativamente equilibrada, su complejidad temporal promedio es (O(nlogn)). Sin embargo, cuando las particiones resultan muy desequilibradas, puede alcanzar una complejidad de (O(n^2)) en el peor caso. La profundidad de la recursión también depende de la estructura de las particiones (Cornell University, 2026).

En la implementación utilizada en este trabajo, QuickSort emplea la partición de Hoare y

selecciona como pivote el elemento ubicado en la posición central del segmento procesado. Por tanto, los resultados experimentales corresponden específicamente a esta variante y no necesariamente representan el comportamiento de todas las implementaciones de QuickSort.

2.4 Complejidad temporal de algoritmos

La complejidad temporal describe cómo aumenta el trabajo requerido por un algoritmo conforme crece la cantidad de elementos de entrada. La notación (O) expresa una cota asintótica y permite estudiar el crecimiento del costo sin depender exclusivamente de los tiempos medidos en una computadora particular.

La Tabla 1 resume las complejidades temporales de los algoritmos estudiados.

Tabla 1. Complejidad temporal de Insertion Sort y QuickSort

Algoritmo	Mejor caso	Caso promedio	Peor caso
Insertion Sort	(O(n))	(O(n^2))	(O(n^2))
QuickSort	(O(nlogn))*	(O(nlogn))	(O(n^2))

Nota. Para QuickSort, el mejor caso indicado corresponde a particiones suficientemente equilibradas; el comportamiento depende de la implementación y de los datos procesados. Elaboración propia a partir de MIT (s. f.) y Cornell University (2026).

2.5 Patrones de entrada y métricas de evaluación

El orden inicial de los elementos puede influir en el trabajo realizado por un algoritmo de ordenamiento. Por esta razón, el experimento considera cinco patrones de entrada: aleatorio, ascendente, descendente, casi ordenado y con valores duplicados. Estos permiten observar el comportamiento de ambos algoritmos bajo distintas distribuciones de los datos.

La evaluación contempla el tiempo de ejecución en milisegundos, las comparaciones registradas, los intercambios, los movimientos y la profundidad máxima de recursión. Estas métricas permiten complementar la medición temporal con información sobre el trabajo realizado durante el ordenamiento. No obstante, los movimientos contabilizados por cada implementación deben interpretarse según sus respectivas definiciones, pues no representan necesariamente operaciones equivalentes.

En conjunto, estos fundamentos permiten establecer una base teórica para interpretar los resultados experimentales y contrastar las tendencias observadas con las complejidades conocidas de ambos algoritmos.